

SELF-PACED SWE MASTERCLASS

GIT GOOD

SWE 101

```
$ git commit -m "getting j*b is not that hard gng 🦄" --no-verify
```

Written & Compiled by Srajan Rai

2026 · First Edition

About the Author and This Book

About the Author

I'm Srajan Rai, a software engineer writing high-performance backend systems in C#/.NET who got deeply tired of finding a j*b in 2026 and wanted to help others avoid the same pitfalls.

Like most developers, I spent years writing code that was completely fine—right up until AI bubble showed up and took my job. Every chapter in this 215-chapter monster exists because I had to prepare for them in these 4 months of my unemployed career. I wanted to understand the raw data, from starting, from zero, why your CPU stalls for 200 cycles when you chase a pointer across the heap, how memory barriers stop hardware reordering, and why microservices always pick 3:00 AM to deadlock.

The standard was simple: if I couldn't explain the physical hardware mechanics, sketch the memory layout, and write the zero-allocation implementation cold on a whiteboard under interview pressure, I didn't actually own the concept. So I wrote the roadmap to fix that, and yes I bought an iPad for this btw xd

About This Book

This is not just an AI slop cheat sheet, and it is not an academic textbook from 1982 written in Assembly Language. It is an unapologetic, exhaustive **215-chapter, 9-phase Poojan Ivl masterclass** engineered to take you from *"it works on my machine and I vibe-coded"* to Staff-level systems mastery and Senior-level design architecture.

We start at the absolute metal—logic gates, binary arithmetic, clock cycles, and stack frame pointers—and systematically build upward through kernel internals, lock-free ring buffers, database storage engines, distributed consensus algorithms, zero-allocation compiler optimizations, and multi-region system design.

Every single chapter follows an unyielding **5-Layer Pedagogical Framework**:

- **1. Physical Memory & Hardware Reality:** What physical RAM, CPU registers, and 64-byte cache lines are actually doing under the hood while your code executes.
- **2. Abstract Theory & Mathematical Invariants:** Formal complexity bounds ($\mathcal{O}$, Ω , Θ) and proof-of-correctness invariants so you stop guessing at algorithmic limits.

- **3. Production-Grade Implementation:** Complete, unabridged source code with zero-allocation optimizations and error handling—no lazy `// TODO: left as an exercise for the reader` shortcuts.
- **4. Elite Algorithmic & LeetCode Patterns:** Core interview archetypes and brutal edge cases designed to ensure you never freeze in a technical screening.
- **5. Verification, Synthesis & Multi-Format Examination:** Self-assessment checklists and multi-question quizzes with full technical rationale so you can audit your own gaps before an interviewer does.

The goal is not just to pass an interview or collect buzzwords for LinkedIn. It is to build mechanical knowledge so sharp that code reviews get better, your latency graphs drop, and you never have to type `git push --force` with your eyes closed again.

Table of Contents

Foundations, Machine Architecture & Core Logic

1. How Computers Execute Code
2. Binary, 2's Complement & Floating-Point Precision
3. Memory Hierarchy: Caches, Stack & Heap
4. Primitive Types, Casting & Memory Sizing
5. Control Flow & Hardware Branch Prediction
6. Loops, Invariants & Misprediction Hazards
7. Functions, Stack Frames & Calling Conventions
8. Parameter Passing Semantics & Stack Allocations
9. Static Arrays, Dynamic Lists & Index Arithmetic
10. Strings, Encodings & Interning Pools
11. Singly & Doubly Linked Lists
12. Stacks, Queues & Circular Buffers
13. Recursion & Call Stack Mechanics
14. Tail Recursion & Frame Reuse (TCO)
15. Hash Tables, Bitwise Masking & Collisions
16. Equality Contracts & GetHashCode Invariants
17. Classes, Structs & Object Header Overhead
18. Inheritance, vtables & Virtual Dispatch
19. Interfaces & Interface Dispatch Tables (itables)
20. Boxing, Unboxing & Generic Specialization
21. Collections & LINQ Pipeline Internals
22. Iterators, yield return & State Machines

- 23. Exception Handling Internals & SEH Unwinding
- 24. Production Debugging & Memory Dumps
- 25. Algorithmic Complexity & Asymptotic Bounds

Intermediate Data Structures & Algorithms

- 26. Two-Pointer Convergence & Fast/Slow Pointers
- 27. Sliding Window Mastery: Fixed & Dynamic
- 28. Prefix Sums & Difference Arrays
- 29. Monotonic Stacks & Monotonic Deques
- 30. Binary Search & Search Space Reduction
- 31. String Searching: KMP, Rabin-Karp & Z-Algorithm
- 32. Priority Queues, Heaps & Flat-Array Math
- 33. Quadratic Sorting Algorithms
- 34. Divide-and-Conquer: Merge, Quick & Heap Sort
- 35. Non-Comparison Sorting: Counting, Radix & Bucket
- 36. Interval Scheduling & Sweep Line Algorithms
- 37. Disjoint Set Union (DSU / Union-Find)
- 38. Trees & Binary Search Trees (BST)
- 39. Tree Traversals: DFS & BFS Level-Order
- 40. Self-Balancing AVL Trees & Rotations
- 41. Red-Black Trees & Production Sets
- 42. Tries, Radix Trees & Suffix Arrays
- 43. Graph Representations: Matrix, List & CSR
- 44. Graph Traversals: Multi-Source & Cycles
- 45. Topological Sorting & DAG Dependencies
- 46. Shortest Paths: Dijkstra & Bellman-Ford

47. All-Pairs Shortest Paths: Floyd-Warshall & Johnson's

48. Minimum Spanning Trees: Kruskal & Prim

49. Tarjan's SCC, Bridges & Articulation Points

50. Network Flow: Ford-Fulkerson & Dinic's

Dynamic Programming & Optimization

51. DP Foundations: Memoization vs. Tabulation

52. 1D Dynamic Programming Patterns

53. 2D DP: Grids, LCS & Edit Distance

54. Knapsack DP: 0/1, Unbounded & Bounded

55. Interval DP & Matrix Chain Multiplication

56. Bitmask DP & State Compression

57. Tree DP & Subtree Aggregations

58. Digit Dynamic Programming

59. Dynamic Programming on DAGs

60. Backtracking Systems: Subsets & Permutations

61. Constraint Satisfaction: N-Queens & Sudoku

62. Greedy Choice & Matroid Theory

63. Segment Trees & Lazy Propagation

64. Binary Indexed Trees (Fenwick Trees)

65. SkipLists, Bloom Filters & HyperLogLog

Low-Level Systems & OS Internals

66. CPU Hardware Architecture & ISAs

67. Instruction Pipelining & Out-of-Order Execution

68. Hardware Branch Prediction & Speculation

69. Cache Line Mechanics & MESI Coherence
70. False Sharing & Cache Line Bouncing
71. SIMD & Vector Registers (AVX / NEON)
72. Virtual Memory Architecture & Paging
73. TLBs, HugePages & MMU Lookups
74. Kernel vs. User Space & System Calls
75. Process Internals, fork & Copy-On-Write
76. Threads, TCBs & OS Scheduling
77. Linux Process Schedulers (CFS) & Pinning
78. Memory Allocators: malloc, jemalloc & Arenas
79. Garbage Collection: Mark-Sweep & Tri-Color
80. Generational GC Mechanics & LOH Compaction
81. Managed Runtime Object Layouts & SyncBlocks
82. File Systems Architecture: Inodes & Ext4/XFS
83. OS Page Cache, Dirty Pages & Direct I/O
84. Synchronous vs. Asynchronous I/O & epoll
85. Linux io_uring & Kernel Bypass
86. Inter-Process Communication (IPC) & Sockets
87. POSIX Signals & Async-Signal-Safety
88. Binary File Formats (ELF/PE) & Dynamic Linking
89. Binary Exploitation Defenses: ASLR, DEP & ROP
90. Unsafe Memory, Pointers & Interop
91. Memory-Mapped I/O (MMIO) & Interrupts
92. Direct Memory Access (DMA), NVMe & SPDK

Concurrency, Lock-Free & Parallelism

- 93. Hardware Memory Models: TSO & Weak Ordering
- 94. Memory Barriers & Acquire-Release Semantics
- 95. Atomic Operations & LOCK Bus Prefixes
- 96. Compare-And-Swap (CAS) & the ABA Problem
- 97. Spinlocks, Ticket Locks & MCS Locks
- 98. Linux Futex & Two-Phase Locking
- 99. Shared vs. Exclusive Reader-Writer Locks
- 100. Coffman Conditions & Deadlock Hierarchies
- 101. Livelock, Starvation & Priority Inversion
- 102. Condition Variables, Monitors & Signalling
- 103. Bounded Buffers & Dining Philosophers
- 104. Lock-Free Treiber Stack Algorithm
- 105. Lock-Free Michael-Scott Queue
- 106. LMAX Disruptor Architecture & Ring Buffers
- 107. Wait-Free Synchronization & Consensus
- 108. Memory Reclamation: EBR & RCU
- 109. Work-Stealing Pools & Chase-Lev Deques
- 110. Asynchronous Continuations & Event Loops
- 111. async/await Compiler State Machines
- 112. Coroutines, Fibers & Green Threads
- 113. Reactive Streams & Backpressure Propagation
- 114. Software Transactional Memory (STM)
- 115. The Actor Model & Message Passing
- 116. Fork-Join Frameworks & Parallel Streams
- 117. GPU Computing Foundations & CUDA

118. CPU-GPU Shared Virtual Memory

Networking, Sockets & Web Architecture

119. Ethernet Framing, MAC & MTU/Jumbo Frames

120. IPv4/IPv6 Headers & CIDR Subnetting

121. IP Routing: BGP, OSPF & Anycast

122. TCP Handshake, Teardown & State Machine

123. TCP Flow Control & Sliding Windows

124. TCP Congestion Control (CUBIC, BBR)

125. TCP Performance Tuning & Nagle's Algorithm

126. UDP Datagrams, Checksums & Multicast

127. QUIC Transport Protocol & Migration

128. BSD Network Socket Programming API

129. High-Throughput I/O Multiplexing (Reactor)

130. Domain Name System (DNS) & DNSSEC

131. Cryptography & TLS 1.3 Handshakes

132. HTTP/1.1 Framing & Chunked Transfer

133. HTTP/2 Multiplexing, Streams & HPACK

134. HTTP/3 over QUIC & QPACK Compression

135. WebSockets, SSE & WebRTC Protocols

136. gRPC, Protocol Buffers & Binary Formats

137. RESTful Architecture & Idempotency Keys

138. OAuth 2.0 & PKCE Security Architecture

139. OpenID Connect (OIDC) & JWT Verification

140. Web Security: CORS, CSRF, XSS & CSP

Database Internals & Storage Engines

- 141. Relational Model & Database Normalization
- 142. ACID Guarantees & Hardware Realities
- 143. Transaction Isolation Levels
- 144. Concurrency Anomalies & Write Skew
- 145. MVCC, Undo Logs & Snapshot Isolation
- 146. Lock Escalation & Two-Phase Locking (2PL)
- 147. Disk Storage Page Layout & Slotted Pages
- 148. Row vs. Column Storage: OLTP vs. OLAP
- 149. B-Tree & B+ Tree Storage Engines
- 150. LSM-Trees, MemTables & SSTables
- 151. Write-Ahead Logging & ARIES Recovery
- 152. Clustered vs. Non-Clustered Indexes
- 153. Composite Indexes & Leftmost Prefix Rule
- 154. Inverted, Bitmap & Sparse Indexes
- 155. Spatial & Vector Indexing (HNSW)
- 156. Volcano Query Execution Engines
- 157. Query Optimizers: CBO & RBO
- 158. Join Algorithms: Hash, Merge & Nested Loop
- 159. Partitioning Strategies & Partition Pruning
- 160. Sharding & Cross-Shard Routing
- 161. Database Replication Topologies
- 162. Replication Lag & Consistency Anomalies
- 163. NoSQL Paradigms: KV, Document & Column
- 164. Time-Series Databases & Gorilla Compression
- 165. Graph Databases & Index-Free Adjacency

166. Distributed SQL: Spanner & CockroachDB

Distributed Systems, Consensus & Cloud

167. Fallacies of Distributed Computing

168. CAP Theorem & PACELC Trade-offs

169. Physical Clocks, NTP & Drift Anomalies

170. Lamport Timestamps & Vector Clocks

171. Hybrid Logical Clocks (HLC) & TrueTime

172. Failure Detectors & Gossip Protocols (SWIM)

173. Distributed Messaging & Idempotent Delivery

174. Distributed Locks & Fencing Tokens

175. Two-Phase Commit & Distributed Transactions

176. The Saga Pattern & Outbox Workflows

177. Consensus Theory & FLP Impossibility

178. Paxos Consensus: Multi-Paxos & Leases

179. Raft Consensus: Election & Log Matching

180. Byzantine Fault Tolerance (BFT) & PBFT

181. Kafka Streaming Logs & Zero-Copy Pipelines

182. Event Sourcing & CQRS Architecture

183. Distributed Caching & Consistent Hashing

184. Redis Architecture & Cluster Topology

185. Layer 4 vs. Layer 7 Load Balancing

186. API Gateways & Dynamic Routing

187. Distributed Rate Limiting Algorithms

188. Resiliency: Circuit Breakers & Full Jitter

189. Service Discovery & Health Probing

- 190. Service Mesh, Envoy & mTLS
- 191. Distributed Tracing & OpenTelemetry
- 192. Container Internals: Namespaces & Cgroups
- 193. OCI Container Runtimes & Isolation
- 194. Kubernetes Control Plane Architecture
- 195. Kubernetes Networking & CNI Plugins
- 196. MicroVMs & Serverless Architectures
- 197. Infrastructure as Code & State Engines
- 198. GitOps, Blue-Green & Canary Deploys

High-Performance, Compilers & Staff+

- 199. CPU Profiling, Flamegraphs & PMU Counters
- 200. Memory Profiling, Dumps & Leak Diagnosis
- 201. Zero-Allocation Spans & Memory Pooling
- 202. Low-Latency Trading & DPDK Bypasses
- 203. High-Performance Binary Serialization
- 204. Compiler Front-End: Lexing & Parsing
- 205. ASTs, Symbol Tables & SSA Form
- 206. Compiler Optimizations & Inlining
- 207. Tiered JIT Compilation & OSR
- 208. Linux eBPF Sandboxing & XDP Bypasses
- 209. Domain-Driven Design (DDD) Core
- 210. Clean & Hexagonal Architecture
- 211. System Design: URL Shorteners & Rate Limiters
- 212. System Design: Collaborative Editors & Chat
- 213. System Design: Video Streaming & Ride-Share

214. System Design: Distributed Checkout Engines

215. SRE, Error Budgets & Staff+ Leadership

How to Use This PDF

This book is built to be written on, annotated, and benchmarked, not just read, ask Gemini about it.

The Margin

Every page is designed with generous margin space. Use it actively: summarize complex invariants in your own words, trace pointer mutations with custom memory diagrams, and jot down performance hypotheses next to algorithms.

Color Legend

- Indigo, Key Idea:** The foundational hardware contract or algorithmic invariant. If you remember one core concept from a section, remember this.
- Orange, Common Misconception:** Latent performance pitfalls, memory leaks, and flawed assumptions that trip up engineers. Pay extra attention here.
- Pink, Interview Angle:** How this exact mechanism is tested during FAANG and high-frequency trading technical screenings.
- Green, Tip / Interview-Ready Answer:** Production takeaways, zero-allocation micro-optimizations, and crisp whiteboard answers.
- Violet, Quiz & Verification:** Deep-dive self-assessment quizzes with comprehensive multiple-choice, writing prompts, and answer keys.

Quizzes & Synthesis Challenges

A rigorous quiz concludes every chapter, and an intensive synthesis challenge closes every phase. Dedicated writing lines are provided below each question. Force yourself to write out the physical hardware mechanics and algorithmic invariants by hand before consulting the small-print answer keys at the bottom.

Pace & Verification Discipline

There is no artificial timeline. Work through the 215 chapters sequentially. Whenever an assembly instruction or memory allocation is discussed, verify it independently using tools like Godbolt Compiler Explorer, SharpLab.io, and BenchmarkDotNet. Consistency and depth always beat speed.

How to Take Notes

I can't write this entire PDF myself so sorry for AI jargons

Restate the Physical Invariant

If a section explains CPU cache line false sharing, do not just copy the definition. Sketch the two CPU cores and annotate the single 64-byte cache line bouncing between their L1 caches in your own handwriting.

Mark Latent Edge Cases & Gaps

Whenever an algorithmic invariant or lock-free edge case feels non-obvious, place a prominent question mark in the margin. Re-read the chapter after writing out the step-by-step state transitions until the concept is crystal clear.

Cross-Reference Across Phases

When studying database storage engines in Phase 7, cross-reference back to B-Tree cache line alignment in Phase 4 and asynchronous I/O in Phase 4. Elite software engineering is about connecting low-level primitives to high-level distributed systems.

Rehearse Spoken Explanations

Answer the Pink Interview boxes out loud without looking at notes. Write your exact spoken explanation in the margin. If you cannot articulate the systems trade-offs cleanly out loud, you do not fully own the concept yet.